

Day 9 — Cloud, Kubernetes, Terraform, delivery, frontend, and DevEx

Outcome

Be able to deploy and operate an AI platform across network, compute, data, Kubernetes, infrastructure code, CI/CD, and user experience with clear ownership and rollback.

Scope note: Day 8 owns service-level MLOps/LLMOps and security behavior. This day owns the cloud, Kubernetes, infrastructure-as-code, delivery, frontend, and developer-experience mechanisms that enforce and expose those requirements.

1. Cloud architecture choices

Compute

- Virtual machine (VM): control and familiar operations; manual scaling/patching burden.
- Container: portable immutable application unit.
- Kubernetes: schedules and manages container workloads.
- Serverless: low operational overhead for suitable stateless/event work, with runtime/latency limits.
- GPU compute: model training/inference, expensive and capacity-sensitive.
- Managed model endpoint: managed serving, less infrastructure control.

Do not choose Kubernetes merely because the system uses AI. Choose from workload diversity, scaling, availability, control, and platform maturity.

Storage

- Block: disk attached to compute/database.
- File: shared filesystem semantics.
- Object: documents, datasets, models, artifacts, backups.
- Relational/NoSQL/vector/cache: chosen from access patterns.

Scalability and availability

- Vertical scaling: larger instance.
- Horizontal scaling: more instances.
- Stateless services scale horizontally more easily.
- High availability uses multiple healthy instances/AZs and failover.
- Disaster recovery restores after major loss.
- RTO: acceptable recovery time.
- RPO: acceptable data loss window.

Autoscaling does not solve provider quotas, slow databases, large model memory, or unbounded demand. Add admission control and application concurrency limits.

Cost

Major drivers:

- GPU/compute;

- model tokens;
- network;
- storage/indexes/logs;
- managed services;
- always-on idle resources.

Controls:

- right-size and autoscale;
- shut down idle development compute;
- batch suitable work;
- cache carefully;
- limit RAG context;
- use model tiers;
- lifecycle old objects;
- attribute cost by tenant/team/project.

IBM Cloud, AWS, Azure, and GCP

All provide the same architectural categories—compute, object storage, databases, networking, identity, Kubernetes, AI/ML, secrets, monitoring, and DevOps—under different service names.

Need	IBM Cloud	AWS	Azure	GCP
Object storage	IBM Cloud Object Storage	S3	Blob Storage	Cloud Storage
Kubernetes	IBM Cloud Kubernetes Service	EKS	AKS	GKE
Container registry	IBM Cloud Container Registry	ECR	Azure Container Registry	Artifact Registry
AI platform	watsonx.ai	SageMaker AI	Foundry/Azure ML	Vertex AI-related services
Secrets	IBM Cloud Secrets Manager	AWS Secrets Manager	Key Vault	Secret Manager

The source notes characterize IBM by hybrid-cloud/governance and watsonx, AWS by breadth/mature infrastructure, Azure by Microsoft enterprise/identity integration, and GCP by data/analytics/Kubernetes/AI. Select from existing estate, regulation, data location, skills, agreements, and required services—not a universal ranking.

2. Secure cloud networking

VPC

- └─ public subnets: load balancer, outbound gateway
- └─ private app subnets: API and workers
- └─ private data subnets: database/cache

- Internet gateway enables public subnet internet routing.

- NAT gateway allows private workloads outbound access without direct inbound exposure.
- Security group is stateful and attached to resources/interfaces.
- NACL is stateless and subnet-level.
- Private endpoints keep supported service traffic off public paths.
- API gateway/load balancer controls entry.
- TLS protects transport.
- IAM controls principal permissions.

Load balancing distributes requests across healthy service instances. At the application layer, health/readiness and connection draining matter as much as the load balancer itself.

Avoid overlapping CIDR ranges when future peering/VPN/transit connectivity is expected.

Use least-privilege workload identities rather than broad node permissions.

3. AWS service placement from the source notes

Service	AI platform use
EKS	Kubernetes control plane and workloads.
ALB	HTTP ingress to Kubernetes services.
RDS/PostgreSQL	Tenants, conversations, metadata, jobs, approvals, audit.
ElastiCache/Redis	Cache, limits, transient state, locks/dedup.
S3	Documents, parsed artifacts, datasets, models, exports.
ECR	Immutable container images.
IAM	Human, CI, Terraform, and pod/workload permissions.
Route 53/ACM	DNS and TLS certificates.

Ingress choices solve different layers:

Entry component	Source-note role
API Gateway	Public API management, authentication, quotas, API keys, routing, and request transformation.
ALB	Layer-7 HTTP host/path routing and TLS termination.
NLB	Layer-4 TCP/UDP routing, high throughput, and lower-level/static-IP needs.

Do not select among them by name alone; start from protocol, routing, API-management, latency, and network requirements.

For browser uploads:

```
browser asks backend
→ backend authenticates/authorizes and creates scoped object key
→ short-lived signed URL
→ browser uploads directly to object storage
→ event starts ingestion
```

The URL inherits creator permissions and must be narrowly scoped.

4. Kubernetes

Objects

- Cluster: the Kubernetes control plane plus worker capacity.
- Node: a worker machine that runs pods.
- Pod: running container unit.
- Deployment: desired replicas and rollout.
- Service: stable network endpoint for pods.
- Ingress: external HTTP routing.
- ConfigMap: non-secret configuration.
- Secret: sensitive configuration object; still needs encryption/access controls.
- HPA: changes pod replicas using metrics.

Probes

- Startup: protect slow initialization.
- Readiness: receive traffic?
- Liveness: restart?

Do not let a temporary external provider outage trigger destructive restart loops.

Resources and scaling

Set requests/limits thoughtfully. AI API, ingestion, evaluation, and tool workers have different resource/concurrency profiles.

Scaling layers:

```
HPA → pod replicas
node autoscaler → cluster capacity
application limits → protect dependencies
provider quota → external ceiling
```

CPU may be a weak signal for I/O-heavy GenAI. Queue depth, active streams, concurrency, or latency can be more representative where available.

Workload separation

Separate:

- interactive API;
- ingestion workers;
- model serving;
- evaluation jobs;
- tool workers.

Benefits include independent scale, fault isolation, and rollout.

Rollouts

- Rolling: efficient gradual replacement.
- Blue/green: parallel environments and fast traffic reversal at higher cost.

Handle streaming connections with graceful shutdown, readiness removal, drain time, and client retry/resume semantics.

5. Helm

- Chart: reusable package/templates.
- Values: configuration inputs.
- Release: installed chart instance.

Use environment-specific values without rebuilding the application image.

```
helm upgrade --install  
→ Kubernetes rollout  
→ smoke/behavior checks  
→ observe  
→ rollback if needed
```

Do not let Helm and Terraform both own the same application resources.

6. Terraform

Terraform is declarative infrastructure management.

Building blocks:

- resource: managed infrastructure.
- data: existing information.
- variable: input.
- locals: derived reusable values.
- output: exposed result.

Lifecycle:

```
init → plan → reviewed apply
```

destroy removes managed infrastructure and requires exceptional care.

State

State maps configuration addresses to real resources and may contain sensitive values.

Team environments use:

- remote durable storage;
- encryption;
- access control;
- locking;
- audit.

The source notes distinguish the older S3 plus DynamoDB lock pattern from native S3 lockfiles for newer designs.

Drift

Drift is out-of-band infrastructure change. Detect with plan, decide whether the change is valid, update code or revert infrastructure, and reduce uncontrolled console changes.

Modules and environments

```
modules/  
  vpc/  
  eks/  
  rds/  
  application/  
environments/  
  dev/  
  stage/  
  prod/
```

Use modules for stable capabilities, not a wrapper per individual resource.

Separate production state and preferably accounts/permissions from lower environments. Workspaces fit similar lower-risk instances but are weaker security boundaries.

Structure state by blast radius and ownership.

Naming conventions

Use consistent names such as:

```
<organization>-<product>-<environment>-<region>-<resource>
```

Tag owner, environment, application, cost center, data classification, and managing system.

7. CI/CD with Jenkins

Pipeline as code:

```
checkout  
→ lint/type/unit tests  
→ security/dependency checks  
→ build immutable image  
→ integration tests  
→ push image by digest/git SHA  
→ deploy development  
→ smoke + AI behavioral evaluation  
→ approval/promotion  
→ deploy staging/production  
→ verify/observe  
→ rollback
```

The Jenkins controller coordinates jobs and configuration; Jenkins agents execute pipeline work. Keep builds isolated, use scoped credentials, and avoid depending on one long-lived mutable build machine.

Build once and promote the same artifact. Do not rebuild different production binaries per environment.

GenAI-specific gates:

- golden RAG cases;
- prompt/tool regression;
- schema compatibility;
- safety/injection cases;
- cost/latency thresholds;
- citation/grounding;
- graph/checkpoint compatibility.

Credentials live in managed CI/workload identity, not the repository or logs.

Rollback:

- previous image digest;
- previous Helm release;
- compatible configuration;
- previous model/prompt/index/graph versions.

8. Ansible

The source notes position:

- Terraform: provision declarative infrastructure.
- Ansible: configure hosts and automate operational tasks.
- Helm: package/deploy Kubernetes applications.

Core Ansible concepts:

Concept	Role
Inventory	Managed hosts and groups.
Playbook	Desired operational workflow.
Task	One action.
Module	Reusable operation with structured behavior.
Handler	Change-triggered action such as restart/reload.
Role	Reusable tasks, handlers, variables, and templates.
Vault	Encrypted storage mechanism for sensitive Ansible data.

Prefer modules to raw shell commands because modules understand desired state, support idempotence, provide structured output, and handle platform differences more safely.

```
- name: Configure Jenkins agents
  hosts: jenkins_agents
  become: true
  tasks:
    - name: Install Docker
      ansible.builtin.apt:
        name: docker.io
        state: present
        update_cache: true
```

Use cases include bootstrapping Jenkins agents/bastions, installing CLI or monitoring tools, rotating configuration, managing certificates on traditional hosts, and standardizing developer VMs.

Trade-offs and pitfalls:

- agentless simplicity depends on SSH/network reachability;
- YAML flexibility can become complex without roles;
- Vault does not necessarily replace a dedicated secrets manager;
- repeated runs should converge rather than repeat effects;
- do not recreate Terraform, configure short-lived Kubernetes pods, or overlap ownership with immutable images/managed services.

9. Frontend and product experience

React versus Next.js

React supplies component UI. Next.js adds an application framework around React with routing and other web-app capabilities. Choose based on product needs rather than the AI backend.

Chat components

- conversation/message list;
- composer;
- streaming answer;
- citations/source drawer;
- feedback;
- error/retry/cancel;
- tool activity and approval;
- upload/ingestion status;
- model/session settings where allowed.

Separate frontend state by ownership:

- server state: conversations, messages, sources, jobs;
- UI state: drawers, selected citation, modal state;
- streaming state: partial answer and event cursor;
- authentication state;
- tenant/environment configuration.

Avoid putting every category into one global store. The backend remains the source of authorization truth.

Streaming

SSE is simpler for one-way server-to-browser token/events. WebSockets fit bidirectional real-time interaction.

Define typed events:

```
{
  "event": "citation",
  "request_id": "req-123",
  "data": {"document_id": "doc-45", "title": "Policy"}
}
```

Design partial failure: an error can happen after some tokens were displayed.

Trust

- display source title, section/page, and version/date;
- let users open the source;
- distinguish official versus lower-authority sources;
- show insufficient evidence;
- represent agent steps/approval without exposing sensitive internals.

The frontend should call a controlled backend, not hold provider secrets or bypass authorization.

Prototype and comparison interfaces

Streamlit and Gradio are useful for internal demos, rapid experiments, stakeholder validation, and model comparisons. A demo UI is not automatically production-ready: enterprise authentication, accessibility, audit, robust state, tenant administration, scalable streaming, and observability may still be required.

A multi-model playground can compare the same prompt across models, latency/cost, retrieval settings, prompt versions, and human preference. Hidden randomized ordering can reduce obvious comparison bias.

Design recoverable errors. Prefer:

| "The CRM tool timed out. No changes were made. Retry the CRM step?"

over an unexplained status code. Preserve user input and expose failed, cancelled, incomplete, and retryable states.

10. Monorepo and developer experience

Possible layout:

```
apps/  
  backend/  
  frontend/  
workers/  
packages/  
  shared-api-types/  
infra/  
  terraform/  
  helm/  
tests/  
docs/
```

Benefits:

- coordinated contracts;
- shared tooling;
- one review/change for cross-stack work.

Costs:

- larger CI graph;
- ownership and release complexity;
- risk of unnecessary coupling.

Developer experience:

- one-command local setup;
- model/tool fakes;
- declared environment configuration;
- API schemas/shared types;
- fast unit tests;
- integration profiles;
- architecture decisions and runbooks;
- reproducible dependencies.

Keep environment settings, Terraform state, Helm values, application configuration, and secret ownership aligned.

A practical repository workflow uses short-lived feature branches, mandatory review, protected main, automated checks, frequent merges, release tags, and feature flags for incomplete behavior. Long-lived environment branches tend to drift; environment differences belong in controlled configuration and promotion, not divergent application code.

11. End-to-end deployment

```
Git push
→ reviewed CI
→ immutable backend/frontend/worker images in ECR
→ Terraform-provisioned network/EKS/RDS/Redis/S3/IAM
→ Helm release to EKS
→ ALB/Ingress/Service/ready pods
→ API connects privately to data/model/tool dependencies
→ smoke and AI-quality gates
→ gradual promotion
→ telemetry and rollback
```

12. Common mistakes

- Public databases or broad security groups.
- Secrets in code/state/logs.
- One production replica.
- State inside ephemeral containers.
- Assuming more pods overcome provider limits.
- CPU-only HPA for queue/concurrency-bound work.
- Liveness checks on unstable external providers.
- Mutable latest image tags.
- Rebuilding per environment instead of promotion.
- Terraform and Helm ownership conflict.
- One state file for excessive blast radius.
- Production applies from developer laptops.
- Dropping active streams during rollout.
- Direct frontend model calls.
- Healthy pods but regressed RAG/tool behavior.
- Monitoring infrastructure only.

Project-grounded examples

Scenario 1: using Ansible for repeatable host and benchmark configuration

Project scenario. In **DPDK Automation for Network Packet Processing**, reusable Ansible roles installed OS packages, gcc/AOCC/clang toolchains, statistics utilities, and benchmark dependencies across Ubuntu 22.04/24.04 and RHEL 8/9. Conditional tasks and variables handled platform differences, while Python/Redfish components handled BIOS-specific operations.

How the concepts apply. This is exactly Ansible's configuration-management role: converge long-lived benchmark hosts toward a repeatable environment and reuse roles across workloads. It also illustrates why module/role boundaries and idempotence matter; repeated campaign setup should not accumulate accidental machine state.

Decision and trade-offs. Shared roles reduced manual variation and made multi-server campaigns feasible, but cross-OS conditions increased role complexity and test combinations. Keeping BIOS automation in dedicated Python/Redfish components

avoided forcing every platform operation into Ansible, at the cost of coordinating two automation layers.

Outcome. The framework became the team’s default execution path, supported multi-server parallel scenarios, and made 10–50+ scenario campaigns practical. The project says setup time and human error were significantly reduced but gives no numeric reduction.

Senior/Staff interview framing.

- **Senior:** explain role inputs, OS condition handling, idempotent reruns, failure reporting, and how a new benchmark consumes shared setup.
- **Staff:** explain ownership boundaries among host configuration, BIOS/platform control, benchmark execution, and reporting; then discuss how you prevented platform variation from fragmenting the whole system.

Scenario 2: production-style delivery for BenchOps Copilot

Project scenario. **DPDK BenchOps Copilot** used FastAPI, Kubernetes, Helm, HPA, Jenkins, Postgres, a vector database, and S3/MinIO. The documented safeguards included tracing, retries, timeouts, circuit-breaker-style protection, CI evaluation gates, and canary/rollback deployment thinking.

How the concepts apply.

```
Jenkins quality/evaluation gates
→ application release
→ Helm-managed Kubernetes deployment
→ HPA-based scaling
→ FastAPI service
→ Postgres + vector database + S3/MinIO dependencies
→ tracing and rollback controls
```

This provides a real example of separating application packaging/deployment from the data systems the application consumes. It also shows why deployment health must include RAG and tool behavior, not only pod health.

Decision and trade-offs. Kubernetes and Helm provided standardized deployment and scaling controls, but added operational overhead that would not be justified for a small one-process prototype. HPA improved elasticity but could not remove downstream model, database, or tool limits. Jenkins evaluation gates improved release confidence at the cost of longer and more specialized CI.

Senior/Staff interview framing.

- **Senior:** describe the build/test/deploy/verify/rollback path and the readiness implications of Postgres, vector, object, model, and tool dependencies.
- **Staff:** justify Kubernetes from workload and operational needs, identify independent scaling/failure domains, define behavior-aware promotion gates, and state which bottleneck HPA cannot solve.

Scenario 3: dashboard and trust-oriented UX

Project scenario. The first project collaborated with a UI/dashboard team on a parameter-driven interface with benchmark settings, statistics toggles, BIOS options, sensible defaults, graphs, metric drill-down, and side-by-side run comparisons. The

Copilot added cited answers and visibility into tool-driven workflows as trust requirements.

How the concepts apply. The UI reduced domain complexity without removing expert control. Defaults and structured controls lowered configuration error; comparisons connected raw automation to an engineer's actual decision. In the AI evolution, citations and explicit operational boundaries were part of usability because benchmark engineers needed to verify advice.

Senior/Staff interview framing.

- **Senior:** explain validation, dependent fields, run status, comparison selection, and how source/citation details help a user recover from uncertainty.
- **Staff:** frame UX as an operational safety surface: defaults, provenance, approval, and clear failure states reduce misuse and support adoption.

Evidence boundary and platform gap. The projects do not name a frontend framework, cloud provider, Terraform, container registry, ingress controller, network topology, Kubernetes probe configuration, rollout mechanism, or exact HPA metric. Do not claim React/Next.js, AWS/Azure/GCP/IBM Cloud, Terraform, EKS/AKS/GKE/IKS, or a specific rollout implementation.

Hypothetical improvement. Terraform-managed cloud infrastructure and explicitly separated Kubernetes workloads could be evaluated if scale, ownership, or environment-reproducibility requirements justified them. This is not documented as implemented.

13. Interview questions

1. VM versus container versus Kubernetes versus managed endpoint?
2. Scale-up versus scale-out?
3. High availability versus disaster recovery; RTO versus RPO?
4. Public versus private subnet?
5. Security group versus NACL?
6. Pod versus Deployment versus Service versus Ingress?
7. Startup versus readiness versus liveness?
8. Why separate API and ingestion workers?
9. HPA versus node scaling versus admission control?
- 10
 - . Rolling versus blue/green?
- 11
 - . Terraform state, drift, modules, and environment isolation?
- 12
 - . Terraform versus Helm versus Ansible?
- 13
 - . Why build once and promote?
- 14
 - . SSE versus WebSockets?
- 15
 - . Why use signed uploads?
- 16
 - . What makes an AI deployment gate different?

17

. How do IBM Cloud, AWS, Azure, and GCP map to the same architecture concepts?

18

. When are Streamlit or Gradio appropriate, and what remains for production?

19

. Cluster versus node versus pod?

14. Exit checklist

- [] Draw secure multi-AZ networking and service placement.
- [] Choose compute/storage/database options.
- [] Explain Kubernetes objects, probes, scaling, and rollouts.
- [] Explain Helm release ownership.
- [] Explain Terraform state, drift, modules, and isolation.
- [] Design immutable CI/CD with AI-quality gates and rollback.
- [] Design streaming, citation, upload, feedback, and approval UX.
- [] Propose a maintainable repository and local workflow.

Source notes

- [Cloud AI Architecture Guide](#)
- [Deploying ML Models API](#)
- [MLOps for AI Systems](#)
- [Databricks Fundamentals](#)
- [Capstone Revision Day 1](#)
- [Capstone Revision Day 3](#)
- [DPDK Automation for Network Packet Processing](#)
- [DPDK BenchOps Copilot](#)